# Chapter 5

# The Technological Framework and Software Stack

## 5.1 Backend Infrastructure and Data Platform Architecture

To operationalize the domain-oriented Data Product model and support interactive analytical workloads within Amadori S.p.A., the enterprise data architecture is undergoing an evolutionary transition from an on-premises relational infrastructure toward a scalable, cloud-native data platform.

The baseline enterprise environment relies primarily on an on-premises Oracle database, which historically serves as the operational transaction repository and the target for analytical queries. To overcome the computational constraints of centralized on-premises storage and eliminate contention between transactional processing and resource-intensive analytical calculations, the target architecture introduces Google BigQuery as the enterprise cloud data warehouse. BigQuery provides a serverless, highly scalable environment based on distributed compute engines and columnar storage, enabling parallelized execution of complex analytical queries over large-scale historical datasets.

Rather than implementing an abrupt cutover, the transition follows a phased migration strategy. In the initial phase, analytical assets and ingestion pipelines continue to read directly from the operational Oracle database. As data models, historical partitions, and ETL routines are progressively refactored and validated, tables are transferred to Google BigQuery. During this transitional period, both database connections remain active simultaneously.

The presence of a centralized Semantic Layer implemented in Looker plays a strategic role in facilitating this dual-connection phase. Because Looker functions via in-database querying rather than data extraction, it can maintain concurrent

dialect connections to both Oracle and BigQuery. Business dimensions and metrics formalized in LookML can initially target tables hosted on the Oracle instance; subsequently, as individual domains complete their migration to BigQuery, the underlying connection references within the LookML models are updated to point to the cloud tables. This decoupling ensures that downstream analytical consumers, executive reports, and AI interfaces remain completely safeguarded from the physical migration process, experiencing no disruption in metric definitions or query behavior while backend storage is shifted.

## 5.2 Formalizing Business Data: The Semantic Layer with Looker and LookML

To operationalize the conceptual definitions established by the enterprise ontology and provide an auditable single source of truth across analytical processes, the architecture implements Looker and its declarative modeling language, LookML. In this stack, Looker does not operate merely as a data visualization layer, but acts as the centralized execution engine for the enterprise Semantic Layer [21]. Looker operates on an in-database query paradigm: analytical data remains stored within the underlying database, while the semantic platform dynamically compiles user interactions, dashboard requests, and API calls into optimized SQL queries executed directly by the target database engine at runtime [23].

This in-database approach plays a crucial role during the transition from the on-premises Oracle database toward Google BigQuery. Currently, the operational material planning pipeline populates the core fact table (`F_MATERIAL_PLANNING`) via a daily scheduled batch job running directly on the Oracle database. While the long-term target architecture aims to migrate these workloads to Google BigQuery, the abstraction provided by LookML allows the organization to manage this shift transparently. Because database connections are configured at the model level, LookML isolates physical connection dialects and schema differences. When tables are migrated to BigQuery in future deployment phases, connection strings and dialect syntax are updated within the model files without altering the exposed business entities, ensuring zero downtime and complete consistency for downstream users and cognitive agents.

### 5.2.1 Architectural Best Practices and Performance Optimization

Designing a high-throughput Semantic Layer requires balancing analytical flexibility with computational efficiency. When modeling complex supply chain dynamics

in LookML, several engineering best practices were applied to minimize query execution overhead and prevent maintenance bottlenecks [11]:

- **Pushing transformations to the backend:** Complex transformations, recursive allocations, and historical rolling aggregations are executed upstream during database batch processing rather than calculated dynamically inside LookML. Looker is dedicated to aggregation, dimensional filtering, and metric definition, preventing resource-intensive row-level computations at query runtime.

- **Denormalization of frequently filtered attributes:** While relational normalization minimizes redundancy in transactional platforms (OLTP), analytical queries run faster when frequently joined descriptive attributes are pre-aggregated or denormalized into wide structures, reducing join depth.

- **Handling surrogate keys and natural codes:** Redundant internal surrogate keys are systematically hidden from the interface. Instead, the model exposes standardized, concatenated business attributes (combining natural alphanumeric codes with their descriptions), streamlining both human analysis and machine interpretation.

- **Minimizing runtime join overhead in dashboard visualizations:** While an Explore model file defines an extensive relational network to grant complete self-service flexibility, individual dashboard tiles must query only the strictly necessary views. Looker generates SQL dynamically based on the dimensions selected in each tile; keeping dashboard queries isolated to minimal join paths directly reduces runtime join operations on the database engine, eliminating computational bottlenecks and query execution delays.

- **Curating the semantic exposure boundary:** Fields that are not explicitly declared in LookML remain completely invisible to consuming tools. Restricting exposed dimensions to certified attributes establishes an architectural boundary that protects users from deprecated columns.

### 5.2.2 Centralized Governance and the DRY Principle in Security Design

A foundational engineering objective of formalizing business semantics in LookML is strict compliance with the *Don't Repeat Yourself* (DRY) principle, which directly prevents code duplication and long-term maintenance overhead across enterprise analytics.

In conventional reporting architectures, enforcing data access segregation between distinct operational entities typically generates architectural anti-patterns. Technical teams are frequently forced to replicate entire models, SQL queries, or dashboard copies simply to hardcode facility-specific filtering criteria (e.g., maintaining cloned reports for each regional logistics deposit) or to strip out sensitive data columns for non-executive audiences. This duplication incurs severe operational risks: whenever an underlying business logic changes the alteration must be manually propagated across every replicated asset, creating high maintenance costs, security vulnerabilities, and eventual metric drift.

Within the LookML framework, operational business definitions and data security boundaries are decoupled, enabling the enterprise to enforce granular data access over a single, unified codebase:

- **Dynamic Row-Level Security (RLS) via `access_filter`:** Rather than creating dedicated Explores or dashboards for different facilities, row-level boundaries are enforced dynamically at query generation time. Through Looker's `access_filter` parameter, the platform maps an authenticated user's session attribute directly to the physical column. When a user runs a query or loads a dashboard, Looker dynamically injects the parameterized filtering predicate into the SQL `WHERE` clause, ensuring strict data compartmentalization across the multi-site supply chain without duplicating a single line of modeling code.

- **Granular Column-Level Security (CLS) and Access Grants:** Field visibility is governed systematically at the semantic declaration level rather than by partitioning underlying database tables. Non-actionable technical fields - such as surrogate identifiers, system timestamps, and internal status flags - are universally obscured from user visibility using `hidden: yes`, preventing clutter in the Explore interface and filtering unnecessary noise from the AI agent's semantic context. Furthermore, to safeguard confidential commercial or financial metrics without generating separate reporting views, LookML provides `required_access_grants`. For instance, while gross material volumes are accessible to all operational planners, a related measure tracking standard component unit costs can be locked behind an access grant tied to an executive or managerial permission set. Users or agents lacking that explicit entitlement are completely prevented from querying the restricted measure or seeing it in their semantic schema, ensuring security by design while maintaining a single, centralized model.

### 5.2.3 Implementation: Material Planning Fact and Master Views

The operationalization of the Material Planning engine described in Chapter 3 is implemented through a set of dedicated LookML views and explores. For readability and editorial conciseness, the following listings present certified excerpts of the production LookML codebase, highlighting the primary fact aggregations and representative master attributes while omitting peripheral metadata and secondary joins.

The primary fact view, `f_material_planning`, defines the daily gross material requirements across forecast dates, finished items, logistics and production platforms, and packaging or ingredient components:

Listing 5.1: LookML Definition for Material Planning Fact View (Partial Excerpt)

```
view: f_material_planning {
  sql_table_name: `gsc-dp-mt-plan-sbx.material_planning_export.
     F_MATERIAL_PLANNING` ;;
  label: "Fabbisogno lordo materiali"

  dimension: primary_key {
    primary_key: yes
    type: string
    sql: concat(cast(${data_mp} as string), '-', cast(${art_id} as
        string), '-',
                cast(${ptf_id} as string), '-', cast(${mat_id} as
                   string), '-',
                cast(${mat_um_id} as string)) ;;
    description: "Chiave primaria univoca: DATA_MP x ART_ID x
       PTF_ID x MAT_ID x MAT_UM_ID"
    hidden: yes
  }

  dimension_group: data_previsione {
    type: time
    label: "Data Previsione"
    timeframes: [raw, date, week, month, quarter, year]
    datatype: date
    convert_tz: no
    sql: ${data_mp} ;;
    description: "Data di previsione del fabbisogno lordo"
    synonyms: ["data di previsione", "data fabbisogno", "orizzonte
        temporale", "data pianificata"]
  }

  measure: fabbisogno_materiale {
    type: sum
    label: "Fabbisogno Materiale"
```

```
29      sql: ${mat_qta} ;;
30      description: "Fabbisogno lordo del materiale (imballo,
            etichetta o ingrediente) necessario a coprire la produzione
             pianificata, per deposito, data e unita di misura"
31      value_format_name: decimal_0
32      synonyms: ["fabbisogno lordo", "quantita materiale", "consumo
            previsto", "fabbisogno totale"]
33      drill_fields: [dettaglio_fabbisogno*]
34    }
35
36    set: dettaglio_fabbisogno {
37      fields: [
38        materiale.tipo_materiale,
39        materiale.materiale,
40        data_previsione,
41        fabbisogno_materiale,
42        um_cod
43      ]
44    }
45
46    measure: casse_articolo {
47      type: sum
48      label: "Casse Articolo"
49      sql: ${art_casse} ;;
50      description: "Casse di prodotto finito previste da cui
            derivano le quantita di materiale"
51      value_format_name: decimal_0
52      synonyms: ["volume casse", "casse previste", "casse
            equivalenti", "casse di prodotto finito", "volume in casse"
            ]
53      drill_fields: [dettaglio_casse*]
54    }
55
56    set: dettaglio_casse {
57      fields: [
58        articolo.articolo,
59        data_previsione,
60        casse_articolo
61      ]
62    }
63  }
```

In this view, technical surrogate keys (`art_id`, `mat_id`, `ptf_id`, `mat_um_id`) and raw numeric dates are suppressed from user visibility using `hidden: yes`. Instead, the view exposes a clean `dimension_group` parsing the date into standard calendar intervals. Business measures—such as `fabbisogno_materiale` and `casse_articolo`—incorporate pre-configured `drill_fields`, enabling users to click on any aggregated volume to immediately inspect the underlying constituent ar-

ticles and material classifications.

Dimensional master views provide the descriptive attributes required to contextualize these fact aggregations. Across the semantic model, master tables fall into two architectural categories: standard single-purpose lookups and polymorphic role-playing dimensions.

Standard master views follow a direct, one-to-one relationship between the physical storage table and the LookML view. A prime example is `l_ptf`, which encapsulates logistics platforms, manufacturing plants, and distribution centers:

Listing 5.2: LookML Definition for Standard Platform Master View (Partial Excerpt)

```
view: l_ptf {
  sql_table_name: `gsc-dp-mt-plan-sbx.material_planning_export.
     L_PTF` ;;
  label: "Piattaforma"

  dimension: ptf_id {
    primary_key: yes
    type: number
    sql: ${TABLE}.PTF_ID ;;
    hidden: yes
  }

  dimension: piattaforma_cod {
    type: string
    sql: ${TABLE}.PTF_COD ;;
    hidden: yes
  }

  dimension: piattaforma_des {
    type: string
    sql: ${TABLE}.PTF_DES ;;
    hidden: yes
  }

  dimension: piattaforma {
    type: string
    label: "Piattaforma"
    sql: concat(${piattaforma_cod}, ' - ',${piattaforma_des}) ;;
    description: "Codice e descrizione del deposito o piattaforma
       logistica"
    synonyms: ["deposito", "stabilimento", "impianto", "sito
       produttivo", "magazzino", "deposito di produzione", "
       deposito di spedizione"]
  }
}
```

As shown in Listing 5.2, standard lookups streamline user consumption: raw identifiers and isolated code/description fields are concealed via `hidden: yes`, exposing a single concatenated business attribute (`piattaforma`) enriched with business synonyms. Symmetrical designs govern other master views.

In contrast, the enterprise item registry, `l_art_jde`, represents a complex polymorphic dimension. In the corporate ERP system (JD Edwards), all physical inventory items—from finished commercial products to raw components—are stored within that single unified master table.

Architecturally, modeling this registry could have followed two competing paradigms:

1. **Dual View Anti-Pattern:** Authoring two distinct LookML view files (e.g., `articolo.view.lkml` and `materiale.view.lkml`) pointing to the identical underlying database table. While conceptually intuitive, this strategy directly violates the *Don't Repeat Yourself* (DRY) principle. Maintaining duplicated codebases introduces severe maintenance risks: any upstream schema update, data type change, or common metadata adjustment must be synchronized manually across multiple files, inevitably leading to code drift and inconsistent business definitions.

2. **Centralized Semantic View with Model-Level Specialization (Implemented Architecture):** Consolidating all physical item attributes and business classifications into a single authoritative LookML view (`l_art_jde`), while delegating operational specialization and aliasing to the model file through role-playing joins.

To enforce the DRY principle and ensure centralized maintainability, `l_art_jde` unifies the entire item schema within a single definition:

Listing 5.3: LookML Master Articles View (Representative Excerpt)

```
view: l_art_jde {
  sql_table_name: `gsc-dp-mt-plan-sbx.material_planning_export.
     L_ART_JDE` ;;
  view_label: "Articolo"

  dimension: art_jde_id {
    primary_key: yes
    type: number
    sql: ${TABLE}.ART_JDE_ID ;;
    hidden: yes
  }

  dimension: articolo_lcod {
    type: string
    sql: ${TABLE}.ART_JDE_LCOD ;;
    hidden: yes
```

```
16    }
17
18    dimension: articolo_des {
19      type: string
20      sql: ${TABLE}.ART_JDE_DES ;;
21      hidden: yes
22    }
23
24    dimension: prodotto {
25      type: string
26      label: "Prodotto Finito"
27      sql: concat(${articolo_lcod}, ' - ',${articolo_des}) ;;
28      description: "Codice e descrizione alfanumerica del prodotto
            finito"
29      synonyms: ["articolo di vendita", "articolo finito", "prodotto
             finito in linea", "PFL", "prodotto finito da neutro", "PFN
            ", "SKU"]
30    }
31
32    dimension: materiale {
33      type: string
34      label: "Materiale"
35      sql: concat(${TABLE}.ART_JDE_LCOD, ' - ',${TABLE}.ART_JDE_DES)
             ;;
36      description: "Codice e descrizione alfanumerica del materiale
            componente"
37      synonyms: ["componente", "materia prima"]
38    }
39
40    dimension: tipo_materiale {
41      type: string
42      label: "Tipo Materiale"
43      sql: ${TABLE}.TIPO_MAT ;;
44      description: "Classificazione del materiale tra imballaggio e
            ingrediente primario"
45      synonyms: ["categoria materiale", "tipologia componente", "
            imballo vs ingrediente"]
46      suggestions: ["Imballo", "Ingrediente"]
47    }
48    dimension: articolo_neutro_cod {
49      type: number
50      label: "Articolo Neutro Cod"
51      sql: ${TABLE}.ART_JDE_NEUTRO_COD ;;
52      description: "Codice dell'articolo neutro, se presente"
53      synonyms: ["codice neutro", "articolo neutro", "codice
            articolo neutro", "prodotto neutro", "PN"]
54    }
55  }
```

As documented in Listing 5.3, `l_art_jde` defines the complete domain dictionary in one location. The actual operational role separation is achieved in the model configuration (5.4) by instantiating this single view twice: once as `articolo` and once as `materiale` (both joined via `art_jde_id`).

### 5.2.4 Model Definition: Relational Joins and Access Perimeter

The relational orchestration of the Material Planning domain is codified within the LookML model file (`material_planning.model.lkml`). The model specifies the active database connection, imports the view definitions, and defines the join topology centered around the primary fact view:

Listing 5.4: LookML Model and Explore Configuration for Material Planning (Simplified Excerpt)

```
connection: "materialplanningbq"

include: "/views/*.view.lkml"

explore: material_planning {
  label: "Fabbisogno lordo materiali"

  access_filter: {
    field: l_ptf.piattaforma_cod
    user_attribute: piattaforma_autorizzata
  }

  join: articolo {
    from: l_art_jde
    view_label: "Articolo"
    type: inner
    relationship: many_to_one
    sql_on: ${f_material_planning.art_id} = ${articolo.art_jde_id}
        ;;
    fields: [articolo.articolo, articolo.articolo_attivo, articolo
       .articolo_neutro_cod]
  }

  join: materiale {
    from: l_art_jde
    view_label: "Materiale"
    type: inner
    relationship: many_to_one
    sql_on: ${f_material_planning.mat_id} = ${materiale.art_jde_id
       } ;;
    fields: [materiale.materiale, materiale.tipo_materiale]
```

```
  }

  join: l_ptf {
    type: inner
    relationship: many_to_one
    sql_on: ${f_material_planning.ptf_id} = ${l_ptf.ptf_id} ;;
  }

  join: l_um {
    type: inner
    relationship: many_to_one
    sql_on: ${f_material_planning.mat_um_id} = ${l_um.um_id} ;;
  }

  join: l_lin {
    type: inner
    relationship: many_to_one
    sql_on: ${articolo.art_jde_lin_id} = ${l_lin.lin_id} ;;
  }

  join: l_fam {
    type: inner
    relationship: many_to_one
    sql_on: ${articolo.art_jde_fam_id} = ${l_fam.fam_id} ;;
  }
}
```

As demonstrated in Listing 5.4, the Explore implements a role-playing dimension by joining the master table `l_art_jde` twice under two distinct semantic contexts: once as `articolo` (representing the finished commercial product generating demand) and once as `materiale` (representing the physical packaging or ingredient consumed). Furthermore, by explicitly filtering exposed attributes via the `fields` parameter, the model exposes only the dimensions relevant to each functional role, achieving clean user-facing specialization while preserving an entirely centralized, DRY-compliant engineering codebase.

### 5.2.5 Semantic Modeling for Self-Service Exploration and AI Enablement

The systematic codification of business rules and enrichment of semantic parameters within LookML serves a dual objective: empowering the network of embedded **Data & AI Champions** with governed self-service intelligence for their operational units, while simultaneously establishing an authoritative, deterministic semantic foundation for autonomous cognitive agents.

**Enabling Governed Self-Service Business Intelligence** In traditional reporting architectures, business users frequently encounter a structural bottleneck: every novel analytical inquiry requires submitting technical ticketing requests to data engineering teams. This dependency introduces operational friction, encourages unmanaged data extraction into local spreadsheets, and inevitably leads to metric fragmentation across corporate silos.

To overcome this operational divide, the operating model formalized in Section 4.6 establishes the network of **Data & AI Champions**—domain specialists embedded directly within operational business units who bridge business requirements with analytical execution. The LookML Semantic Layer serves as the primary technical enabler for these Champions, providing an intuitive, guardrailed canvas that delegates self-service intelligence to their respective functions without compromising governance.

Empowered by Looker Explores, Data & AI Champions drive self-service analytics across their departments through three core operational capabilities:

- **Autonomous ad-hoc investigation:** Within the unified `material_planning` explore, the underlying relational schema is abstracted into certified business-oriented dimensions and pre-aggregated measures. Because join topologies, cardinalities, and filtering predicates are compiled and enforced upstream by LookML, Champions can rapidly model functional inquiries with mathematical confidence, fully protected against malformed joins or invalid Cartesian products.

- **Diagnostic root-cause analysis via `drill_fields`:** Beyond static reporting, Champions leverage pre-configured `drill_fields` on core measures (such as `fabbisogno_materiale` and `casse_articolo`) to perform interactive diagnostic drill-downs. When identifying anomalous demand fluctuations, they can instantaneously decompose the high-level metric into constituent packaging categories, item codes, and daily scheduling buckets, resolving operational bottlenecks autonomously.

- **Decentralized dashboard authoring and peer distribution:** Acting as analytical reference points, Champions translate the daily operational needs of their unit into curated analytical views (*Looks*) and assemble cross-filtered operational dashboards for their immediate peers. Because these domain-specific assets are anchored to centralized LookML metadata, all departmental dashboards automatically inherit upstream logic updates, fully relieving central data engineering from routine reporting debt while safeguarding metric consistency.

**Enabling Grounded Generative AI Agents** While human analysts and Data & AI Champions leverage this semantic framework through visual drag-and-drop interfaces, the exact same declarative metadata acts as an auditable, deterministic API for external computational systems and Generative AI agents [10]:

- **Vocabulary disambiguation via synonyms:** Natural language queries rarely match exact database column names. The `synonyms` parameter maps common enterprise jargon, informal acronyms, and domain terms directly to validated LookML dimensions, allowing an AI agent to match user intent with deterministic schema fields.
- **Semantic anchoring via descriptions:** The `description` parameter defines the exact operational scope and mathematical logic of each metric. This metadata prevents language models from misinterpreting raw values or fabricating fictitious aggregation logic.
- **Context pruning via hidden fields:** By hiding raw IDs and technical join keys, the Semantic Layer filters out noise from the agent's context window. The LLM evaluates only certified, business-relevant fields, substantially reducing prompt token consumption and preventing hallucinated joins.

By encapsulating both business logic and contextual metadata directly within version-controlled code, Looker acts as a unified compiler bridging human operational decision-making with automated analytics. The operational orchestration of this semantic layer with autonomous conversational agents, diagnostic reasoning loops, and dynamic dashboard routing is formally developed in 6.

## 5.3 Governance Roadmap: Extending Lineage and Enterprise Knowledge via OpenMetadata

While Looker operates as the centralized semantic compiler and enforces metric governance at the analytical layer, comprehensive visibility across the broader enterprise data lifecycle requires a dedicated metadata management platform. Currently, business definitions, dimensional attributes, and operational descriptions are codified and maintained directly within LookML files using parameters such as `description`, `label`, and `synonyms`. While this approach successfully establishes LookML as an executable "Business Glossary as Code", it remains primarily accessible to analytics engineers and technical practitioners working within the version-controlled modeling environment.

To decouple business glossary governance from technical codebases and achieve automated data lineage across heterogeneous systems, the enterprise architecture envisions the integration of OpenMetadata as an evolutionary roadmap milestone. OpenMetadata is an open-source, schema-first metadata management platform built upon standard JSON schemas for data discovery, governance, and quality profiling. Within the planned roadmap, OpenMetadata will expand enterprise governance beyond the analytical consumption boundary, focusing on three foundational capabilities:

1. **Centralized, Low-Code Business Glossary:** Transitioning business definitions from plain-text LookML parameters into an intuitive enterprise catalog accessible to non-technical business stakeholders [5]. This provides domain stewards across planning, supply chain, and commercial divisions with the tooling required to manage term ownership, certify metric definitions, and approve terminology changes collaboratively without committing code to a Git repository.

2. **Automated End-to-End Data Lineage:** Establishing systemic traceability across the entire data lifecycle. By connecting directly to source catalogs - tracing tables from their operational origin in the on-premises Oracle database, through the cloud landing zones and transformations in Google BigQuery, down to the views, dimensions, and measures compiled by Looker. Automated lineage tracking will enable data engineering teams to perform proactive impact analysis, accurately identifying which downstream Explores, operational dashboards, or predictive routines will be disrupted prior to executing any upstream schema migration.

3. **Enterprise Knowledge Graph Construction:** Structuring enterprise metadata entities - including relational storage assets, batch pipeline schedules, semantic metric definitions, and user access policies - as an interconnected knowledge graph [15]. This structural context provides deterministic insights into data provenance and reliability, serving as an authoritative grounding foundation to enhance future autonomous AI workflows.

In this target framework, OpenMetadata will ingest metadata programmatically from Looker and the target data platform instances via dedicated metadata APIs, eliminating manual synchronization overhead. This evolutionary enhancement strictly decouples operational query execution (orchestrated in-database by Looker across BigQuery and transition-phase databases) from systemic metadata governance, completing the structural transition toward enterprise-grade Data Products.

In summary, the technological infrastructure deployed at Amadori S.p.A. establishes a resilient migration bridge, facilitating a secure and phased transition from legacy on-premises transactional environments toward the analytical scalability of Google BigQuery. By establishing Looker as an in-database Semantic Layer, the enterprise insulates operational workflows from physical database cutovers, enforces the DRY principle across its metric catalog, guarantees granular access governance, and encapsulates complex supply chain definitions within certified LookML models. With the storage evolution, relational modeling, and metadata governance roadmap formalized, the subsequent chapter explores how this unified semantic layer enables autonomous conversational analytics through Gemini in Looker.

# Bibliography


[1] Christine L. Borgman. *Big Data, Little Data, No Data: Scholarship in the Networked World*. MIT Press, Cambridge, MA, 2015.

[2] Geoffrey C. Bowker and Susan Leigh Star. *Sorting Things Out: Classification and Its Consequences*. MIT Press, Cambridge, MA, 1999.

[3] George E. P. Box and Gwilym M. Jenkins. *Time Series Analysis: Forecasting and Control*. Holden-Day, San Francisco, CA, 1970.

[4] Danah Boyd and Kate Crawford. Critical questions for big data: Provocations for a cultural, technological, and scholarly phenomenon. *Information, Communication & Society*, 15(5):662–679, 2012.

[5] DAMA International. *The DAMA Guide to the Data Management Body of Knowledge (DAMA-DMBOK2 Revised)*. DAMA International, revised edition, 2024.

[6] Thomas H. Davenport and Jeanne G. Harris. *Competing on Analytics: The New Science of Winning*. Harvard Business Review Press, Boston, MA, updated edition, 2017.

[7] Zhamak Dehghani. *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media, Sebastopol, CA, 2022.

[8] Devoteam. Conversational analytics in looker: Architecture, guardrails, and implementation, 2026. Accessed: 2026-10-15.

[9] Darren Edge, Ha Trinh, Aidan Newman, Joshua Christopher, Andrew Evans, Shenghao Wang, Kevin Larson, Robert Foster, Kyle Bennett, Scott Jones, et al. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[10] Google Cloud. Best practices for configuring conversational analytics and ai agents in looker, 2024. Accessed: 2026-10-15.

[11] Google Cloud. Lookml and explore best practices for high-performance queries, 2024. Accessed: 2026-10-15.

[12] Google Cloud. Create and manage explore data agents in looker conversational analytics, 2026. Accessed: 2026-10-15.

[13] Thomas R. Gruber. A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2):199–220, 1993.

[14] Joseph Hilger, Lulit Tesfaye, and Zachary Wahl. *Bridging Knowledge, Data, and AI: Harnessing the Semantic Layer Framework to Drive Intelligence*. Professional and Applied Computing Series. Springer Nature, Cham, Switzerland, 1st edition, 2026.

[15] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d'Amato, Gerard de Melo, Claudio Gutierrez, José Emilio Labra Gayo, Sabrina Kirrane, Sebastian Neumaier, Axel Polleres, Roberto Navigli, Axel-Cyrille Ngonga Ngomo, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan Sequeda, Steffen Staab, and Antoine Zimmermann. Knowledge graphs. *ACM Computing Surveys*, 54(4):1–37, 2021.

[16] Rob Kitchin. *The Data Revolution: Big Data, Open Data, Data Infrastructures and Their Consequences*. SAGE Publications, London, 2014.

[17] Sabina Leonelli. *Data-Centric Biology: A Philosophical Study*. University of Chicago Press, Chicago, IL, 11 2016.

[18] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

[19] Ikujiro Nonaka and Hirotaka Takeuchi. *The Knowledge-Creating Company: How Japanese Companies Create the Dynamics of Innovation*. Oxford University Press, New York, NY, 1995.

[20] Natalya F. Noy and Deborah L. McGuinness. Ontology development 101: A guide to creating your first ontology. Technical Report KSL-01-05, Stanford Knowledge Systems Laboratory, Stanford, CA, 2001.

[21] Dip Bharatbhai Patel. Comparative analysis of data visualization tools: Tableau, power bi & looker. *The Eastasouth Journal of Information System and Computer Science*, 1(1):55–59, 2023.

[22] Pranitha Potturi. Demystifying semantic layers in business intelligence platforms. *Sarcouncil Journal of Engineering and Computer Sciences*, 4(8):612–618, 2025.

[23] James Richardson, Kurt Schlegel, Rita Sallam, Austin Kronz, and Julian Sun. Magic quadrant for analytics and business intelligence platforms. Technical Report G00467317, Gartner, February 2021.